

Plan Directeur

Stage de Fin d'Etudes dominante : Big Data & IA

API d'ordonnancement, d'estimation et de répartition de charge métier

Tuteur de l'entreprise	Stagiaire	Tuteur EIGSI
CHOKRI Soumia	DJOUROBI OMANDA Aldrin Bruno Junior	Zakaria El makhlouki

SOMMAIRE

I. Contexte du Projet	4
1. Contexte général.....	4
2. Diagnostic de l'existant	5
a. Analyse des dysfonctionnements majeurs	5
b. Positionnement stratégique.....	6
3. Présentation de la solution : un système multi-agents générateur de solveurs.....	6
a. Vision du projet	6
b. Valeur ajoutée attendue	6
II. Enjeux.....	8
1. Enjeux qualitatifs.....	8
2. Enjeux stratégiques	8
3. Enjeux économiques	8
4. Enjeux techniques	9
III. Périmètre & Contraintes	10
1. Périmètre du projet.....	10
a. Ce que le projet inclut.....	10
b. Ce que le projet exclut	10
2. Contraintes	11
a. Contraintes temporelles	11
b. Contraintes métiers et humaines.....	11
c. Contraintes de confidentialité et de conformité	11
IV. La Mission et les Objectifs	12
1. Nature de la mission.....	12
2. Objectifs S.M.A.R.T.	12
a. Objectif principal	12
b. Objectifs opérationnels et livrables	12
3. Résultats attendus et livrables	14
V. Analyse Fonctionnelle.....	15
1. Analyse fonctionnelle externe	15
a. Bête à cornes.....	15
b. Diagramme pieuvre	16
VI. Architecture du Système Multi-Agents	19
1. Chaîne de traitement et rôle des agents	19
2. Le modèle pivot d'ordonnancement	20
3. Choix technologiques.....	20

VII. Structuration du Projet (WBS / RBS / OBS).....	22
1. Work Breakdown Structure (WBS) -- Macro-taches	22
2. Resource Breakdown Structure (RBS).....	24
3. Organisationnel Breakdown Structure (OBS).....	24
VIII. Planning de Déroulement.....	25
1. Phases et jalons clés	25
IX. Plan d'Organisation.....	27
1. Matrice RACI.....	27
2. Organisation des réunions	27
3. Jalons clés.....	28
4. Gestion de la documentation et des modifications.....	28
X. Analyse des Risques.....	29
1. Risques projet.....	29
2. Risques techniques	29

I. Contexte du Projet

1. Contexte général

Dans toute organisation industrielle, une question revient chaque jour : comment ordonnancer les tâches de production, estimer leur durée et équilibrer la charge sur les ressources disponibles machines, postes, opérateurs pour tenir les délais sans surcharger ni laisser de capacité inutilisée ? L'ordonnancement est un problème combinatoire difficile, et il varie d'un atelier à l'autre : un atelier d'usinage, une ligne d'assemblage et une unité de conditionnement n'ont ni les mêmes gammes, ni les mêmes contraintes, ni les mêmes objectifs.

Dans la grande majorité des PME et ETI industrielles, cet arbitrage reste manuel : un responsable ouvre un fichier Excel, estime à vue la durée des opérations, observe qui est chargé, qui ne l'est pas, affecte les tâches à la main et ordonnance selon son expérience. Cette pratique provoque des déséquilibres de charge, des délais estimés au jugement, et une mémoire opérationnelle (durées réelles, performances, aléas) inexploitée qu'elle réside dans des fichiers Excel, des exports ERP/MES ou de simples journaux papier.

Le marché propose des solutions d'ordonnancement (APS, MES, OR-Tools applicatifs), mais elles partagent une même limite : le moteur d'optimisation est codé une fois pour toutes, puis paramétré. Les durées d'opérations sont généralement saisies manuellement ou issues de gammes figées ; elles ne sont ni prédites ni recalibrées dynamiquement à partir des données réellement disponibles dans l'entreprise. Adapter un tel moteur à un nouveau processus industriel — nouvelles gammes, nouvelles contraintes, nouvel objectif exige une intervention d'ingénierie longue et coûteuse.

La question centrale de ce projet devient donc : comment concevoir un système multi-agents, exposé via une API, capable — à partir des données disponibles sur le processus de l'entreprise (modèle BPMN lorsqu'il existe, exports ERP/MES, fichiers Excel de suivi, historiques d'exécution), de ses ressources et de ses contraintes — de comprendre le contexte, d'estimer la durée des opérations et la charge prévisionnelle par ressource, d'affecter les tâches aux ressources les plus pertinentes, de générer automatiquement un solveur d'ordonnancement sur mesure, de l'exécuter, puis de superviser le fonctionnement en continu pour proposer des améliorations ?

2. Diagnostic de l'existant

Ce diagnostic s'appuie sur l'observation des pratiques courantes d'ordonnancement industriel, telles qu'elles transparaissent dans les outils de gestion quotidiens (Excel, ERP, gammes papier). Il révèle, au-delà des limites opérationnelles classiques, une rigidité structurelle des outils existants face à la diversité des processus et des formats de données

a. Analyse des dysfonctionnements majeurs

Problématique	Constat	Conséquence
Estimation au jugement	Durées des opérations annoncées de mémoire ou issues de gammes figées, sans recalibration sur les performances réelles.	Ecart significatifs entre charge estimée et charge réelle. Engagements clients non tenus.
Répartition manuelle	L'affectation des tâches aux ressources se fait sans critère objectif ni vision globale de la charge.	Ressources inadaptées mobilisées, compétences mal exploitées, déséquilibres récurrents.
Ordonnancement au jugement	L'ordre de passage et l'affectation sont décidés de mémoire, sans optimisation des gammes ni des capacités réelles.	Goulots non anticipés, makespan dégradé, délais tenus au prix d'heures supplémentaires.
Moteur figé, non adaptatif	Les outils encodent un modèle fixe ; tout changement de processus ou de contrainte exige une reparamétrisation lourde.	Coût d'adaptation élevé, délais d'intégration longs, solution non transposable.
Mémoire opérationnelles dormante	Durées réelles, aléas et performances sont enregistrés mais jamais réinjectés dans la planification.	Aucune amélioration continue ; les mêmes erreurs se répètent ; actif data non valorisé.
Pilotage rétrospectif	Les indicateurs de charge et de retard sont calculés en fin de période.	Vision tardive : impossible de corriger une dérive pendant qu'elle se produit.

Tableau 1 : Analyse des dysfonctionnements

b. Positionnement stratégique

Une étude comparative confronte la gestion actuelle (Excel + jugement) aux standards du marché : suites APS/MES industrielles, modules d'ordonnancement d'ERP, et bibliothèques d'optimisation génériques. Ces solutions présentent trois limites face à une exigence d'adaptabilité : cout et délai d'adaptation élevés, spécialisation par déploiement non transposable, et opacité du moteur freinant l'adhésion des équipes terrain.

C'est précisément cet écart entre la richesse des solveurs disponibles et le cout de leur adaptation à chaque contexte, conjugué à l'absence de tout mécanisme d'estimation et de répartition intelligente que ce projet entend combler par une couche d'intelligence qui estime, répartit et génère le solveur plutôt que de le figer.

3. Présentation de la solution : un système multi-agents générateur de solveurs

a. Vision du projet

Le projet ne vise pas à livrer un nouveau moteur d'ordonnancement de plus, mais une couche d'intelligence multi-agents qui fabrique le bon moteur pour chaque contexte. À partir des **données disponibles sur le processus d'une entreprise industrielle , modèle BPMN, export ERP/MES, fichier de suivi Excel ou historique d'exécution** , de ses ressources et de ses contraintes, le système comprend le contexte, estime la durée des opérations et la charge prévisionnelle par ressource, affecte les tâches aux ressources les plus pertinentes, formalise le problème, génère un solveur sur mesure, l'exécute, puis supervise le fonctionnement réel pour proposer des améliorations continues.

Le différenciateur central : ni l'ingestion, ni les estimations, ni les affectations, ni le solveur ne sont codés en dur. Le système s'adapte à la source de données disponible — qu'elle soit un BPMN structuré ou un simple export Excel — et normalise systématiquement ces données vers un modèle pivot unique, à partir duquel les estimations et les affectations sont générées et recalibrées à la demande, en fonction des gammes, des ressources et des contraintes effectives de l'atelier. Le système agit comme un ingénieur-ordonnanceur augmenté, capable de lire un processus quelle que soit sa forme, d'en estimer la charge, d'affecter intelligemment les tâches, de déduire un modèle d'optimisation, d'écrire le code correspondant, de le valider, puis de l'exploiter.

b. Valeur ajoutée attendue

On passe d'un ordonnancement subi (manuel, au jugement, figé, dépendant d'un format de données unique) à un ordonnancement piloté et adaptatif : ingestion flexible des données

existantes de l'entreprise, estimations calibrées sur l'historique réel, répartition fondée sur des critères objectifs, charge équilibrée, délais fiabilisés, et une solution qui se reconfigure d'elle-même lorsque le processus — ou la source de données — change, sans nouveau projet d'intégration. La supervision continue transforme la mémoire opérationnelle dormante, quel que soit son format d'origine, en boucle d'amélioration.

II. Enjeux

La mise en place d'un système d'ordonnancement adaptatif pilote par agents répond simultanément à des enjeux de qualité, de stratégie, d'économie et de technique.

1. Enjeux qualitatifs

- Précision des estimations : passer de durées annoncées au jugement à des estimations calibrées sur l'historique réel et la charge prévisionnelle calculée par ressource.
- Pertinence de la répartition : affecter chaque tâche à la ressource la plus adaptée (compétences, disponibilité, charge courante) avant de séquencer.
- Optimalité des ordonnancements : produire des séquences optimisées sur des objectifs explicites (malaire, retard total, utilisation des ressources).
- Fiabilisation des délais : remplacer les engagements approximatifs par des délais issus d'un calcul traçable et justifiable.
- Reproductibilité et explicabilité : garantir qu'à situation comparable le système propose la même solution, justifiée et auditable.

2. Enjeux stratégiques

- Adaptabilité multi-contextes industriels : un même socle d'agents capable de produire estimations, répartitions et solveurs pour des ateliers très différents, sans réécriture du cœur applicatif.
- Capitalisation sur la donnée et le processus : transformer le modèle BPMN et les logs d'exécution en actifs directement exploitables pour piloter et améliorer la production.
- Pilotage en continu : grâce à l'agent de supervision, détecter les dérives et proposer des correctifs avant qu'ils ne deviennent visibles cote client.
- Acculturation à l'IA agentique : démontrer la capacité de l'organisation à intégrer des architectures multi-agents dans ses processus opérationnels.

3. Enjeux économiques

- Réduction du coût d'intégration : supprimer le projet d'ingénierie spécifique à chaque atelier c'est le système qui génère estimations, répartitions et solveur adaptés.
- Libération du temps d'ingénierie et de management : récupérer le temps consacré à l'estimation manuelle, à l'affectation et à la re-paramétrisation des outils.

- Réduction des coûts de non-qualité : limiter retards, pénalités, heures supplémentaires non anticipées et sous-traitance d'urgence.

4. Enjeux techniques

- Fiabilité du code génère : le solveur produit doit être systématiquement valide (faisabilité, respect des contraintes) avant exécution.
- Qualité des estimations : les durées prédites doivent être suffisamment précises pour que la répartition et l'ordonnancement résultants soient exploitables.
- Performance et scalabilité : calcul suffisamment rapide pour un usage opérationnel, avec séparation nette entre génération (hors ligne) et exécution (en ligne).
- Intégration transparente : déploiement par conteneurisation, exposition en API REST, compatible avec le SI existant sans refonte.
- Conformité et sécurité : anonymisation des données, chiffrement des communications, exécution maîtrisée (sandbox) du code génère.

III. Périmètre & Contraintes

1. Périmètre du projet

a. Ce que le projet inclut

- Comprendre le processus : ingestion et analyse des données disponibles sur l'atelier industriel pilote — modèle BPMN, exports ERP/MES, fichiers Excel de suivi ou historiques d'exécution — ainsi que de ses ressources et de ses contraintes.
- Estimer les charges : prédiction de la durée de chaque opération (estimation unitaire) et calcul de la charge prévisionnelle par ressource sur l'horizon de planification.
- Repartir les tâches : affectation de chaque tâche à la ressource la plus pertinente (compétences, disponibilité, charge courante) avant séquençement.
- Formaliser le problème : construction d'un modèle pivot d'ordonnancement (jobs, opérations, ressources, contraintes, objectifs) indépendant de l'atelier.
- Générer un solveur sur mesure : production automatique d'un modèle d'optimisation exécutable (OR-Tools CP-SAT) adapté au contexte, avec validation automatique.
- Exécuter et équilibrer : résolution produisant un ordonnancement et un équilibrage de charge sur les ressources disponibles.
- Superviser et améliorer : boucle de supervision détectant dérives et goulots, et proposant des améliorations.
- Transmettre : documentation permettant à l'entreprise d'exploiter et de faire évoluer le système après le stage.

b. Ce que le projet exclut

- Refonte des processus métiers : le système s'adapte à la façon de produire de l'atelier, il ne la réorganise pas.
- Remplacement du jugement humain : le système propose, le responsable décide ; aucun ordonnancement n'est imposé automatiquement.
- Evaluation des personnes : l'outil aide à ordonnancer et repartir, il ne sert pas à juger les performances individuelles.
- Industrialisation à grande échelle : le livrable est un démonstrateur valide sur un cas pilote, pas un produit critique prêt au déploiement massif.
- Pilotage MES temps réel critique : la supervision reste analytique et consultative, sans prise de contrôle automatisée des équipements.

2. Contraintes

a. Contraintes temporelles

- Durée du stage : l'ensemble de l'analyse du besoin a la démonstration finale doit tenir dans les 26 semaines allouées au stage de fin d'études.
- Disponibilité des interlocuteurs : les phases d'analyse, de validation et de prise en main dépendent de la disponibilité de la tutrice entreprise et des utilisateurs métier.
- Délais d'accès : l'accès aux modèles de processus, aux historiques et aux bonnes personnes doit être anticipé dès le démarrage.

b. Contraintes métiers et humaines

- Acceptation par les utilisateurs : le système doit être perçu comme une aide, jamais comme un instrument de contrôle ou de surveillance.
- Lisibilité pour les non-spécialistes : estimations, répartitions et ordonnancements doivent rester compréhensibles par les responsables d'atelier.
- Confiance dans les sorties générées : estimations, affectations et solveurs doivent être explicables et auditable, condition d'adhésion à une solution qui s'auto-génère.

c. Contraintes de confidentialité et de conformité

- Sensibilité des données : modèles de processus, données de production et performances sont confidentiels et ne doivent pas transiter par des services extérieurs non maîtrisés.
- Respect du RGPD : anonymisation et minimisation des données pouvant inclure des informations personnelles (opérateurs).
- Exécution maîtrisée du code : tout code génère est exécuté en environnement isolé (sandbox), trace et auditable.

IV. La Mission et les Objectifs

1. Nature de la mission

La mission relevé d'une démarche d'ingénierie complète : il ne s'agit pas seulement de produire un système agentique, mais de livrer une solution utilisable, qui apporte une valeur opérationnelles réelle a un atelier industriel. Elle se décompose en trois volets :

- Volet métier : comprendre l'ordonnancement actuel de l'atelier pilote, modéliser son processus (BPMN), formaliser ses contraintes et valider la pertinence de la démarche avec les utilisateurs.
- Volet IA & optimisation : concevoir les agents (compréhension, estimation, répartition, formalisation, génération, validation, exécution, supervision), articuler raisonnement à base de LLM et optimisation déterministe (OR-Tools).
- Volet applicatif : encapsuler le tout dans une API réutilisable, documentée, testée, conteneurisée et déployable.

2. Objectifs S.M.A.R.T.

a. Objectif principal

Concevoir, développer et démontrer un système multi-agents, expose via API, capable d'estimer la durée des opérations et la charge prévisionnelle par ressource, d'affecter intelligemment les taches aux ressources les plus pertinentes, de générer automatiquement un solveur d'ordonnancement et d'équilibrage de charge sur mesure à partir du contexte d'un atelier industriel (BPMN, ressources, contraintes), de l'exécuter, et de superviser le fonctionnement pour proposer des améliorations continues en transformant un ordonnancement manuel et empirique en un processus pilote, reproductible et auditable.

b. Objectifs opérationnels et livrables

Objectif 1 : Compréhension du besoin et du processus.

- Objectif : cartographier le processus de l'atelier pilote, le formaliser en BPMN et formuler le problème dans un format pivot réutilisable.
- Livrable : dossier d'analyse fonctionnelle valide, comprenant la représentation structurée du processus pilote et le cahier des charges fonctionnel.

Objectif 2 : Agent de compréhension du processus.

- Objectif : produire un agent capable de parser un BPMN et d'en extraire une représentation structure exploitable (taches, flux, rôles, ressources).

- Livrable : module d'ingestion fonctionnel, avec rapport de couverture des constructions et formats supportés.

Objectif 3 : Agent d'estimation des charges.

- Objectif : produire un agent capable de prédire la durée de chaque opération (estimation unitaire) et de calculer la charge prévisionnelle par ressource sur l'horizon de planification, à partir du contexte BPMN et de l'historique d'exécution.
- Livrable : agent d'estimation fonctionnel avec rapport d'évaluation de la précision (écart estimation vs réel) sur le cas pilote.

Objectif 4 : Agent de répartition des charges.

- Objectif : produire un agent capable d'affecter chaque tâche à la ressource la plus pertinente (compétences, disponibilités, charge prévisionnelle) avant séquençage, avec un gain mesurable sur la répartition manuelle de référence.
- Livrable : agent de répartition fonctionnel avec rapport de benchmark (comparaison répartition agent vs répartition manuelle).

Objectif 5 : Modèle pivot et agent de formalisation.

- Objectif : définir un modèle pivot d'ordonnancement générique et l'agent qui traduit un contexte concret (issu de l'estimation et de la répartition) vers ce modèle.
- Livrable : spécification du modèle pivot + agent de formalisation, avec jeux d'exemples couvrant plusieurs configurations d'atelier.

Objectif 6 : Génération automatique de solveur (cœur du projet).

- Objectif : générer automatiquement, à partir du modèle pivot enrichi, un solveur exécutable (OR-Tools CP-SAT) adapté aux contraintes, avec un taux de code valide conforme à un seuil cible défini en début de phase.
- Livrable : agent de génération + agent de validation/test, avec rapport sur le taux de génération valide et la qualité des solveurs produits.

Objectif 7 : Exécution, équilibrage et benchmark.

- Objectif : exécuter les solveurs générés et mesurer la qualité de l'ordonnancement (makespan, retard total, utilisation ressources) face à une Baseline (ordonnancement manuel ou règles SPT/EDD).
- Livrable : module d'exécution + rapport de benchmark démontrant un gain mesurable sur la Baseline.

Objectif 8 : Supervision et amélioration continue.

- Objectif : mettre en place l'agent de supervision détectant dérives et goulots à partir des données d'exécutions et proposant des améliorations.
- Livrable : agent de supervision fonctionnel avec scénarios de détection et propositions documentées.

Objectif 9 : Mise à disposition par API.

- Objectif : exposer l'ensemble en API REST conteneurisée et documenter tous les Endpoint.
- Livrable : API documente (Swagger), conteneurisée, avec scénario complet de bout en bout valide.

3. Résultats attendus et livrables

- La solution opérationnelle : démonstrateur multi-agents déployé sur l'infrastructure de test, accessible par API.
- Le code source : référentiel structure, commente et documente, avec instructions de déploiement.
- Les artefacts génèrent : exemples de solveurs génères, modèles pivot, rapports d'estimation et de répartition, jeux de tests de validation.
- La documentation technique : dossier d'architecture du système multi-agents et documentation détaillée du service.

V. Analyse Fonctionnelle

Avant la conception technique, l'analyse fonctionnelle cadre ce que le système doit faire et ce qu'il ne doit pas faire.

1. Analyse fonctionnelle externe

a. Bête à cornes

- A qui rend-il service ? : aux organisations industrielles responsables d'atelier, chefs de production, ordonnanceurs, intégrateurs SI.
- Sur quoi agit-il ? : sur les tâches/gammes de production, la charge des ressources, le modèle de processus (BPMN) et l'historique d'exécution de l'atelier.
- Dans quel but ? : estimer les charges, répartir les tâches, générer un solveur d'ordonnancement adapte, équilibrer la charge et améliorer en continu.

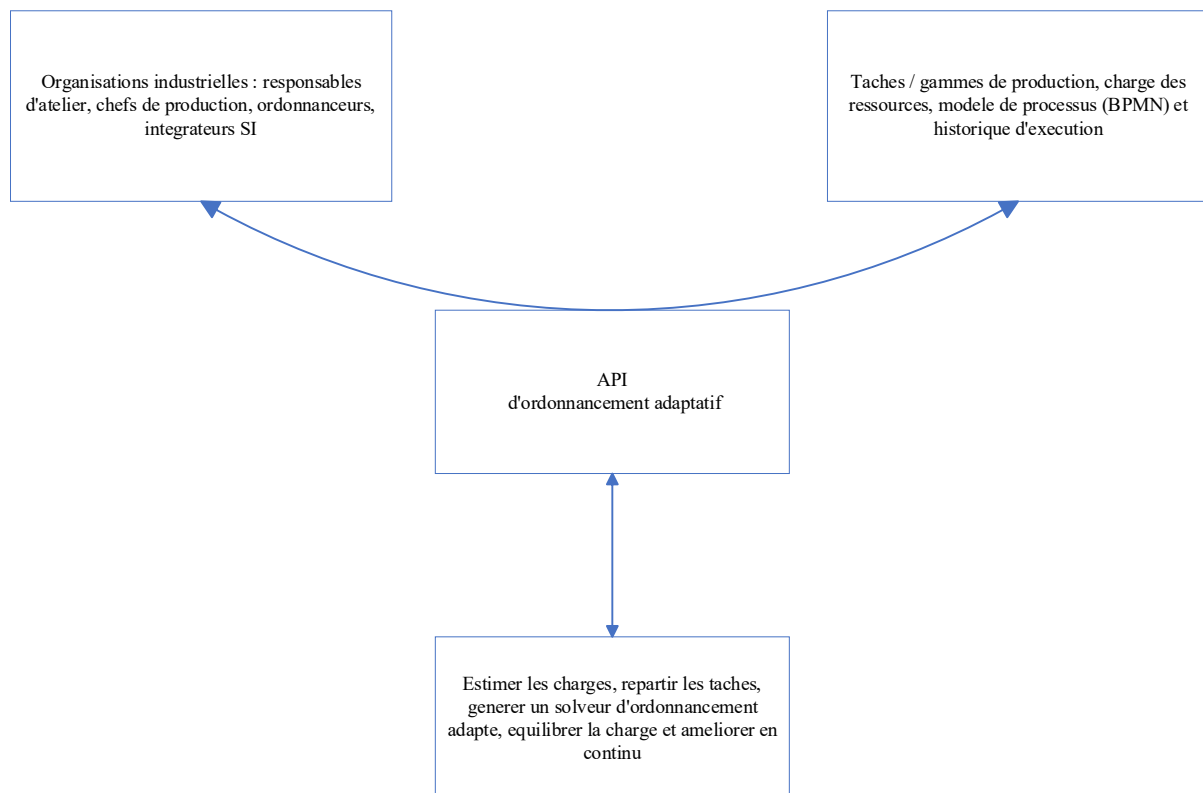


Figure 1 : Diagramme bête à cornes — Système multi-agents d'ordonnancement adaptatif

b. Diagramme pieuvre

Le système est au centre d'un écosystème. On identifie les Fonctions Principales (FP) et les Fonctions Contraintes (FC) reliant le système a ses milieux extérieurs : utilisateurs métiers, applications métiers (ERP/MES), modèles de processus BPMN, historiques d'exécution, ressources et contraintes, infrastructure IT, et services LLM.

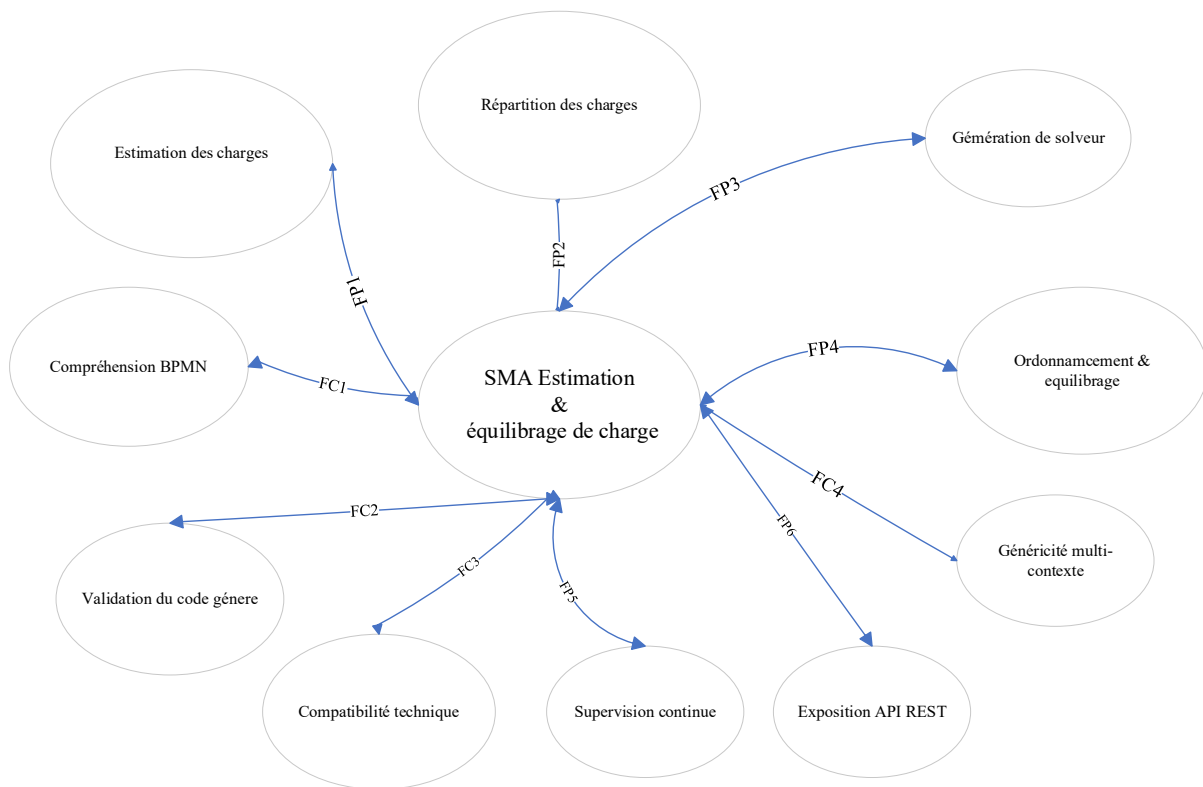


Figure 2 : Diagramme Pieuvre

Ref.	Type	Intitule / Description	Acteurs & outils	Interaction
FP1	Principale	Compréhension du contexte : ingérer et interpréter les données disponibles de l'atelier (BPMN, exports ERP, fichiers Excel de suivi) pour en extraire une représentation structurée (tâches, flux, rôles, ressources)	Agent d'estimation, LLM, historiques d'exécution	Le système retourne une représentation structurée du processus, normalisée quelle que soit la source d'entrée.

FP2	Principale	Répartition des charges : affecter chaque tâche à la ressource la plus pertinente (compétences, disponibilités, charge courante) avant séquençement.	Agent de répartition, ressources, contraintes T-R-C-O	Le système produit une affectation tâche-ressource argumentée, soumise à validation.
FP3	Principale	Génération de solveur : formaliser le problème et générer un solveur d'ordonnancement sur mesure à partir du contexte BPMN, des estimations et de la répartition.	Agent formalisation, agent génération, OR-Tools CP-SAT, LLM	Le système produit un solveur exécutable valide et adapté aux contraintes de l'atelier.
FP4	Principale	Ordonnancement & équilibrage : exécuter le solveur génère pour produire l'ordonnancement et l'équilibrage de charge.	Agent d'exécution, OR-Tools	Le système retourne un planning optimisé avec indicateurs (malaire, retards, utilisation).
FP5	Principale	Supervision continue : observer le fonctionnement, détecter dérives/goulots, proposer des améliorations.	Agent de supervision, données d'exécution	Le système alerte et propose des re-ordonnements ou ajustements en continu.
FP6	Principale	Exposition API REST : mettre tous les services à disposition de toute application via une interface standardisée.	FastAPI, OpenAPI/Swagger, ERP/MES	Les applications externes interrogent le SMA par requêtes HTTP documentées.
FC1	Contrainte	Compréhension BPMN : ingérer et interpréter des modèles BPMN 2.0 hétérogènes.	Parseur BPMN (bpmn-python), agent de compréhension	Tout BPMN 2.0 standard est traduit en représentation structurée.
FC2	Contrainte	Validation du code généré : garantir faisabilité et respect des contraintes avant exécution.	Agent de validation, sandbox, jeux de tests	Aucun solveur non valide n'est exécuté en production.
FC3	Contrainte	Compatibilité technique : s'exécuter sur l'infrastructure existante sans refonte du SI.	Docker, Docker Compose, serveurs cibles	Le système est livré conteneurisé et déployable en une commande.

FC4	Contrainte	Généricité multi-contextes industriels : fonctionner pour des ateliers varies via le modèle pivot.	Modèle pivot, agent de formalisation	Le pivot isole les agents amont des agents aval pour garantir la portabilité.
------------	-------------------	--	--------------------------------------	---

Tableau 2 : Cahier de spécification fonctionnel

VI. Architecture du Système Multi-Agents

Le cœur du projet est une architecture multi-agents orchestrée, exposée via API. Chaque agent porte une responsabilité unique ; ils communiquent par un état partage structure autour du modèle pivot. Deux familles d'agents cohabitent : des agents de raisonnement à base de LLM (compréhension, estimation, répartition, formalisation, génération, supervision) et des composants déterministes (validation, exécution OR-Tools). Ce choix est structurant : le LLM ne résout jamais le problème combinatoire il génère le modèle ; la résolution reste confiée à un solveur exact, garantissant rigueur et reproductibilité.

1. Chaîne de traitement et rôle des agents

La chaîne de traitement s'articule en neuf étapes séquentielles avec boucle de retour :

- **Agent de Compréhension du Processus** : ingère et interprète les données disponibles sur l'atelier (modèle BPMN 2.0, exports ERP/MES ou fichiers Excel de suivi selon la source fournie par l'entreprise), en extrait une représentation structurée commune et identifie les opérations, leurs enchaînements et les ressources mobilisées indépendamment du format d'origine
- **Agent Ressources & Contraintes** : formalise les ressources (machines, postes, opérateurs, calendriers, capacités, compétences) et les contraintes (précédences, temps de réglage, fenêtres de livraison, priorités, indisponibilités).
- **Agent d'Estimation des Charges** : à partir de la structure BPMN extraite et de l'historique d'exécution disponible, prédit la durée de chaque opération (estimation unitaire par apprentissage ou raisonnement) ET calcule la charge prévisionnelle par ressource sur l'horizon de planification. Ces estimations enrichissent le modèle pivot avant toute affectation.
- **Agent de Répartition des Charges** : à partir des estimations et des ressources disponibles, affecte chaque tâche à la ressource la plus pertinente selon des critères objectifs (compétences requises, disponibilités calendaire, charge prévisionnelle courante). Cette étape réduit l'espace de recherche du solveur et garantit que chaque tâche est orientée vers la bonne ressource avant séquençement.
- **Agent de Formalisation** : traduit le processus, son contexte, les estimations et les affectations vers le modèle pivot d'ordonnancement représentation générique et indépendante de l'atelier (jobs, opérations, ressources, contraintes, fonction objectif).

- Agent de Génération de Code : À partir du modèle pivot enrichi, génère un solveur exécutable (modèle OR-Tools CP-SAT) sur mesure, encodant explicitement les contraintes et l'objectif du contexte considéré.
- Agent de Validation & Test : vérifie automatiquement le solveur génère : compilation/exécution en sandbox, faisabilité, respect des contraintes sur jeux de tests synthétiques. En cas d'échec, renvoie un diagnostic à l'agent de génération pour correction (boucle de réparation).
- Agent d'Exécution / Optimisation : exécute le solveur valide sur les données réelles et produit l'ordonnancement et l'équilibrage de charge, accompagnés d'indicateurs (makespan, retards, utilisation).
- Agent de Supervision : à partir des données d'exécution réelles, détecte dérives, goulots et écarts au plan, et propose des améliorations (re-séquencement, ajustement d'estimations, régénération du solveur).
- Orchestrateur : coordonne la séquence des agents, gère l'état partagé, arbitre les boucles de réparation et expose les services au travers de l'API.

2. Le modèle pivot d'ordonnancement

Le modèle pivot est la pierre angulaire de la généricité : c'est une représentation formelle et neutre d'un problème d'ordonnancement enrichi des estimations et affectations (ensemble de jobs et d'opérations avec durées prédites, ressources affectées avec capacités et compétences, contraintes de précédence et de séquencement, fenêtres temporelles, fonction objectif). Tout atelier industriel est d'abord traduit vers ce pivot ; la génération de solveur opère ensuite uniquement à partir du pivot. Cette séparation rend les agents amont (compréhension, estimation, répartition) indépendants des agents aval (génération, exécution), et c'est elle qui permet la portabilité d'un atelier à l'autre.

3. Choix technologiques

- Compréhension BPMN : parsing BPMN 2.0 (bpmn-python / lxml), éventuellement pm4py pour l'analyse de processus ; lecture de fichiers Excel/exports ERP (pandas, openpyxl) selon le format disponible chez le client.
- Estimation des charges : apprentissage supervisé (scikit-learn, XGBoost) sur historiques d'exécution, ou raisonnement LLM en l'absence de données historiques suffisantes.

- **Répartition des charges** : optimisation combinatoire légère (Hungarian algorithm, min-cost flow) ou raisonnement LLM selon la complexité de l'atelier.
- **Agents & orchestration** : framework agentique (LangGraph — référentiel principal, CrewAI / AutoGen en alternative), agents de raisonnement appuyés sur un service LLM.
- **Optimisation** : OR-Tools CP-SAT pour l'ordonnancement (job-shop, flow-shop, RCPSP), avec métaheuristiques (DEAP) en repli si besoin.
- **Génération & validation** : génération de modèles Python OR-Tools, exécution en sandbox isolée, harnais de tests automatisés (pytest).
- **Service & déploiement** : FastAPI, OpenAPI/Swagger, Docker, Docker Compose.
- **Stack 100 % open-source** : écosystème Python, dépôt Git, outils projet (Trello, draw.io)

VII. Structuration du Projet (WBS / RBS / OBS)

Le projet, où se mêlent compréhension métiers, IA agentique et développement applicatif, est structuré selon une approche hybride à trois axes.

1. Work Breakdown Structure (WBS) -- Macro-taches

MT1 Cadrage et analyse du besoin :

Objectif : comprendre l'atelier pilote, modéliser son processus (BPMN), formaliser le périmètre et valider l'approche.

- Entretiens utilisateurs, cartographie et modélisation BPMN, analyse fonctionnelle, cahier des charges, choix d'architecture.
- Méthodes : entretiens semi-directifs, ateliers de cadrage, BPMN, UML. Jalon J1 : Cadrage valide.

MT2 Compréhension & modèle pivot :

Objectif : construire l'agent de compréhension BPMN et le modèle pivot d'ordonnancement.

- Parsing BPMN, extraction structurée, définition du modèle pivot, agent de formalisation, jeux d'exemples.
- Méthodes : bpmn-python, pm4py, agents LLM. Jalon J2 : Modèle pivot valide.

MT3 Estimation des charges :

Objectif : construire l'agent d'estimation capable de prédire la durée des opérations et la charge prévisionnelle par ressource.

- Exploration et préparation des historiques d'exécution, sélection de l'approche (ML supervisée ou raisonnement LLM), entraînement/évaluation, intégration dans le modèle pivot.
- Méthodes : scikit-learn, XGBoost, raisonnement LLM, métriques d'évaluation (MAE, MAPE). Jalon J3 : Agent d'estimation fonctionnel.

MT4 Répartition des charges :

Objectif : construire l'agent de répartition capable d'affecter chaque tâche à la ressource la plus pertinente avant séquençage.

- Modélisation du problème d'affectation, implémentation des critères (compétences, disponibilité, charge prévisionnelle), benchmark vs répartition manuelle.
- Méthodes : algorithme hongrois, min-cost flow, raisonnement LLM. Jalon J4 : Agent de répartition fonctionnel.

MT5 Génération & validation de solveur :

Objectif : générer automatiquement des solveurs OR-Tools sur mesure et les valider.

- Agent de génération de code, agent de validation, sandbox, boucle de réparation, métriques de validité.
- Méthodes : OR-Tools CP-SAT, LLM, pytest, exécutions sandbox. Jalon J5 : Génération de solveur fonctionnel.

MT6 Exécution, équilibrage & benchmark :

Objectif : exécuter les solveurs et mesurer la qualité face à une Baseline.

- Module d'exécutions, calcul des indicateurs, benchmark sur cas de taille croissante vs Baseline manuelle et règles SPT/EDD.
- Méthodes : OR-Tools, métriques (makespan, tardiness, utilisation). Jalon J6 : Gain mesure sur Baseline.

MT7 Supervision & amélioration continue :

Objectif : détecter dérives et goulots, proposer des améliorations.

- Agent de supervision, scénarios de détection, propositions de régénération.
- Méthodes : analyse des logs d'exécutions, KPIs, agents LLM. Jalon J7 : Supervision opérationnelle.

MT8 Mise à disposition par service (API) :

Objectif : encapsuler les agents dans un service réutilisable et conteneurise.

- Architecture du service, Endpoint, authentification, documentation interactive, conteneurisation, tests.
- Méthodes : FastAPI, OpenAPI/Swagger, Docker, pytest. Jalon J8 : API opérationnelle.

MT9 Validation, portabilité & clôture :

Objectif : démontrer la valeur sur le cas pilote, finaliser les livrables académiques.

- Démonstration pilote avec Key User, documentation technique et utilisateur, rapport de stage, soutenance.

- Méthodes : scenarios de démonstration, transfert de connaissance. Jalon J9 : Clôture du projet.

2. Resource Breakdown Structure (RBS)

- Ressources humaines : stagiaire (porteur principal), tutrice entreprise (cadrage et validation métiers), tuteur EIGSI (suivi pédagogique), Key Users (validation fonctionnelle), support IT.
- Ressources matérielles : poste de travail, machine de calcul (CPU/GPU), serveur de test, espace de stockage, accès réseau entreprise.
- Ressources logicielles : Python, LangGraph, service LLM, scikit-learn/XGBoost, OR-Tools/DEAP, Bpmn-python/pm4py, FastAPI, Docker, VS Code, Git, Trello, draw.io.
- Ressources documentaires & données : modèles BPMN et historiques d'exécutions fournis par l'entreprise, jeux synthétiques d'ordonnancement, documentation officielle des outils, articles académiques.

3. Organisationnel Breakdown Structure (OBS)

- Comité de Pilotage (COPIL) : Tutrice entreprise + Tuteur EIGSI. Valide les jalons majeurs et arbitre les évolutions de périmètre.
- Stagiaire (chef de projet) : pilote opérationnel, planification, réalisation des livrables, reporting.
- Tutrice entreprise : cadre les attentes métier, valide les livrables opérationnels, débloque les accès.
- Tuteur EIGSI : suit la conformité aux attendus pédagogiques, valide les livrables académiques.
- Utilisateurs métier (Key Users) : apportent l'expertise terrain en cadrage et validation.
- Support IT entreprise : intervient pour les accès, la configuration et la sécurité.

VIII. Planning de Déroulement

La réalisation s'étend sur 26 semaines. Le planning suit une approche hybride : la composante IA/data suit le cycle CRISP-DM ; la composante développement applicatif suit une approche Kanban ; les phases de cadrage et de clôture suivent une logique séquentielle.

1. Phases et jalons clés

N°	Phase / Tâche / Jalon	Durée	Début	Fin	Prédécesseur
P1	PHASE 1 : CADRAGE & ANALYSE	3 sem.	29/04/2026	19/05/2026	
1.1	Analyse besoin & modélisation BPMN de l'atelier pilote	S1-S2	29/04/2026	12/05/2026	
1.2	Rédaction des spécifications (cahier des charges)	S2-S3	06/05/2026	19/05/2026	1.1
1.3	Architecture multi-agents & choix d'outils	S3	13/05/2026	19/05/2026	1.2
J1	Jalon J1 : Cadrage validé	1 jour	19/05/2026	19/05/2026	1.3
P2	PHASE 2 : COMPRÉHENSION & MODÈLE PIVOT	3 sem.	20/05/2026	09/06/2026	J1
2.1	Agent de compréhension BPMN (parsing, extraction)	S4-S5	20/05/2026	02/06/2026	J1
2.2	Définition du modèle pivot & agent de formalisation	S5-S6	27/05/2026	09/06/2026	2.1
J2	Jalon J2 : Modèle pivot validé	1 jour	09/06/2026	09/06/2026	2.2
P3	PHASE 3 : ESTIMATION DES CHARGES	3 sem.	10/06/2026	30/06/2026	J2
3.1	Exploration et préparation des historiques d'exécution	S7	10/06/2026	16/06/2026	J2
3.2	Agent d'estimation (prédiction durée + charge prévisionnelle)	S7-S9	10/06/2026	30/06/2026	3.1
J3	Jalon J3 : Estimation fonctionnelle	1 jour	30/06/2026	30/06/2026	3.2
P4	PHASE 4 : RÉPARTITION DES CHARGES	2 sem.	24/06/2026	07/07/2026	3.2 (partiel)
4.1	Agent de répartition (affectation tâche-ressource)	S9-S10	24/06/2026	07/07/2026	3.2 (partiel)
4.2	Benchmark répartition agent vs répartition manuelle	S10	01/07/2026	07/07/2026	4.1
J4	Jalon J4 : Répartition fonctionnelle	1 jour	07/07/2026	07/07/2026	4.2
P5	PHASE 5 : GÉNÉRATION & VALIDATION DE SOLVEUR	5 sem.	08/07/2026	11/08/2026	J3, J4
5.1	Agent de génération de solveur (OR-Tools CP-SAT)	S11-S13	08/07/2026	28/07/2026	J3, J4
5.2	Agent de validation, sandbox & boucle de réparation	S13-S15	22/07/2026	11/08/2026	5.1
J5	Jalon J5 : Génération fonctionnelle	1 jour	11/08/2026	11/08/2026	5.2
P6	PHASE 6 : EXÉCUTION, ÉQUILIBRAGE & BENCHMARK	2 sem.	05/08/2026	25/08/2026	5.1 (partiel)
6.1	Module d'exécution & indicateurs	S15-S16	05/08/2026	18/08/2026	5.1 (partiel)

6.2	Benchmark vs baseline (manuel, SPT/EDD)	S16-S17	12/08/2026	25/08/2026	6.1
J6	Jalon J6 : Gain mesuré	1 jour	25/08/2026	25/08/2026	6.2
P7	PHASE 7 : SUPERVISION	2 sem.	19/08/2026	08/09/2026	6.1 (partiel)
7.1	Agent de supervision : détection & propositions	S17-S19	19/08/2026	08/09/2026	6.1 (partiel)
J7	Jalon J7 : Supervision OK	1 jour	08/09/2026	08/09/2026	7.1
P8	PHASE 8 : SERVICE & API	3 sem.	02/09/2026	22/09/2026	J5, J6
8.1	Architecture du service, endpoints, authentification	S19-S20	02/09/2026	15/09/2026	J5, J6
8.2	Documentation Swagger, conteneurisation, tests	S21-S22	16/09/2026	22/09/2026	8.1
J8	Jalon J8 : API opérationnelle	1 jour	22/09/2026	22/09/2026	8.2
P9	PHASE 9 : VALIDATION & CLÔTURE	4 sem.	23/09/2026	27/10/2026	J8, J7
9.1	Démonstration cas pilote + documentation	S23-S24	23/09/2026	06/10/2026	J8
9.2	Rapport de stage & préparation de la soutenance	S24-S26	30/09/2026	27/10/2026	9.1
J9	Jalon J9 : Clôture du projet	1 jour	27/10/2026	27/10/2026	9.2

Tableau 3 : Phases, durées et jalons. Les dates calendaires seront callées sur la date effective de démarrage du stage.

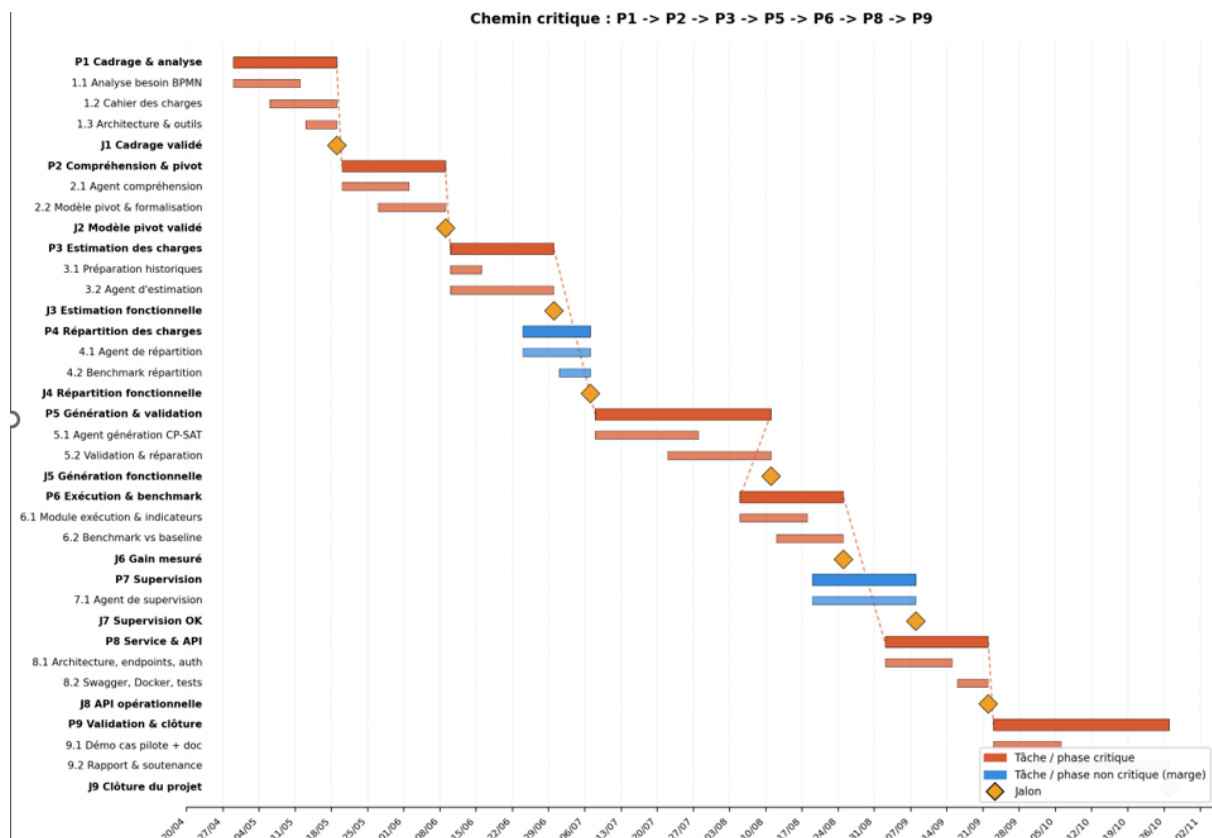


Figure 3 : Diagramme de Gant

IX. Plan d'Organisation

1. Matrice RACI

Activité / Livrable	Stagiaire	Tutrice Entr.	Tuteur EIGSI	Key Users	Support IT
Plan directeur	R	C	A	C	I
Cahier des charges & BPMN pilote	R	A	I	C	--
Architecture multi-agents	R	A	C	--	C
Agent compréhension BPMN & modèle pivot	R	A	I	C	--
Agent d'estimation des charges	R	A	C	C	--
Agent de répartition des charges	R	A	C	C	--
Génération & validation de solveur	R	A	C	I	C
Exécution & benchmark	R	A	C	C	--
Agent de supervision	R	A	C	C	--
API / Service	R	A	I	I	C
Documentation technique & utilisateur	R	A	I	C	--
Rapport de stage & soutenance	R	C	A	--	--

Tableau 4 : Matrice RACI

2. Organisation des réunions

Type de réunion	Fréquence	Participants	Objectif
Point hebdo	1 / semaine	Stagiaire + Tutrice entreprise	Suivi avancement, validation tâches, planification, identification des blocages.
Revue de fin de phase	A chaque jalon	Stagiaire + Tutrice entreprise	Démonstration des livrables, feedbacks, validation formelle du jalon.
Point école	Bimensuel	Stagiaire + Tuteur EIGSI	Suivi pédagogique, alignement aux attendus académiques.
COPIL	Fin de phase majeure	Tutrice entreprise + Responsable service	Validation jalons majeurs, arbitrage des évolutions de périmètre.
Point blocage technique	Au besoin	Stagiaire + Support IT	Résolution de problèmes (accès serveur/données, bug bloquant).
Atelier Key Users	Selon besoin	Stagiaire + Tutrice + Key Users	Recueil du besoin, démonstration intermédiaire, validation fonctionnelle.

Tableau 5 : Plan Organisationnel des réunions

3. Jalons clés

- J1 Cadrage valide : cahier des charges et BPMN pilote valides, architecture définie. Accord formel sur le périmètre.
- J2 Modèle pivot valide : agent de compréhension BPMN et modèle pivot opérationnels. Un BPMN pilote traduit correctement vers le pivot.
- J3 Estimation fonctionnelle : agent d'estimation produisant des prédictions de durée et une carte de charge prévisionnelle. Précision conforme au seuil cible.
- J4 Répartition fonctionnelle : agent de répartition affectant les tâches aux ressources avec gain mesurable sur la répartition manuelle de référence.
- J5 Génération de solveur fonctionnel : agents de génération et de validation produisant un solveur valide. Taux de code valide conforme au seuil cible.
- J6 Gain mesure sur Baseline : exécutions et benchmark documentés. Amélioration mesurable (mésentente/décalage/utilisation) sur la Baseline.
- J7 Supervision opérationnelle : agent de supervision détectant dérives et proposant des améliorations. Scénarios de détection valides.
- J8 API opérationnelle : service conteneurisé et documenté. Scénario complet (BPMN -> estimation -> répartition -> solveur -> ordonnancement -> réponse) sans bug bloquant.
- J9 Clôture du projet : démonstration pilote, documentation complète, rapport et maintenance. Validation COPIL et maintenance EIGSI réussie.

4. Gestion de la documentation et des modifications

- Documentation technique : code commenté, README sur Git, documentation API auto-générée (Swagger), dossier d'architecture du SMA.
- Documentation utilisateur : guide de démarrage rapide, manuel d'intégration pour développeurs tiers.
- Documentation projet : comptes rendus de réunions, liste des points ouverts, journal de bord, dans un espace partagé accessible aux tuteurs.

Toute demande de modification du périmètre suit un processus formel : enregistrement dans la liste des points ouverts, évaluation d'impact par le stagiaire, puis validation par la tutrice entreprise (et le tuteur EIGSI si un livrable académique est concerné).

X. Analyse des Risques

Chaque risque est rattaché à une macro-tâche du WBS, évalue selon une matrice Probabilité x Impact (P et I de 1 à 5 ; criticité $C = P \times I$), et associe à un palliatif. La fiabilité du code génère et la précision des estimations constituent les risques structurants du projet.

1. Risques projet

ID	Risque identifié	MT	P	I	Plan d'action (palliatif)
RP-01	Retard d'accès aux modèles de processus / historiques d'exécutions.	MT1, MT2, MT3	3	5	Anticipation administrative dès la S1 ; repli sur BPMN et jeux synthétiques pour avancer en parallèle.
RP-02	Difficulté à modéliser le processus pilote en BPMN exploitable.	MT2	3	4	Ateliers avec Key User ; périmètre BPMN restreint aux constructions essentielles ; itérations courtes.
RP-03	Historiques d'exécutions insuffisants pour l'agent d'estimation.	MT3	3	4	Approche mixte ML + raisonnement LLM en l'absence de données. Jeux de données synthétiques en repli.
RP-04	Glissement de périmètre (demandes hors initial).	Toutes	3	3	Back log valide à chaque revue ; tout ajout passe par le processus formel de gestion des modifications.
RP-05	Indisponibilité des Key Users / de la tutrice pour validations.	MT1, MT9	3	3	Planification anticipée des créneaux ; validations asynchrones quand possible.
RP-06	Retard de rédaction du rapport / préparation soutenance.	MT9	2	4	Rédaction au fil de l'eau des MT1 ; trame de rapport établie tôt ; jalons intermédiaires de rédaction.

Tableau 6 : Risque projet identifiés

2. Risques techniques

ID	Risque identifié	MT	P	I	Plan d'action (palliatif)
RT-01	Code génère non fiable (modèle infaisable, contrainte mal traduite, hallucination).	MT5	4	5	Agent de validation obligatoire + boucle de réparation + harnais de tests + exécutions sandbox ; Template de génération contraintes ; documentation transparente des limites.
RT-02	Précision des estimations insuffisante (écarts durée estimée vs réelle trop élevés).	MT3	3	4	Benchmark multi-approches (ML supervise vs raisonnement LLM). Validation par les Key Users sur le cas pilote. Documentation transparente des limites et intervalles de confiance.
RT-03	Agent de répartition produisant des affectations sous-optimales.	MT4	3	3	Benchmark compare plusieurs stratégies d'affectation. Validation humaine systématique avant exécutions.

RT-04	Solveur génère peu performant (temps de calcul excessif).	MT5, MT6	3	4	Garde-fous de complexité ; limites de temps ; repli sur métaheuristiques (DEAP) ; benchmark sur tailles croissantes.
RT-05	Couverture BPMN insuffisante (constructions non supportées).	MT2	3	3	Périmètre de constructions BPMN explicite ; tests de couverture ; extension incrémentale.
RT-06	Cout / latence / dépendance au service LLM.	MT3- MT5, MT7	3	3	Mise en cache des générations ; abstraction du fournisseur LLM ; modèles de repli (Olema) ; budgets d'appels surveillés.
RT-07	Solutions peu explicables (rejet par les utilisateurs).	MT4, MT6, MT7	2	3	Justification des estimations, répartitions et ordonnancements (indicateurs, contraintes actives) ; visualisations lisibles ; validation humaine.
RT-08	Sécurité de l'exécution du code génère / non-conformité RGPD.	MT5, MT8	2	4	Exécution exclusivement en sandbox isolée ; anonymisation et minimisation des données ; traçabilité complète.

Tableau 7 : Risques Techniques identifiées